Threshold Signatures as-a-Service: Achieving Threshold ML-DSA in One Online Round

Matthieu Rambaud[1], Sascha Roth[1], and Antoine Urban[2]

[1] Télécom Paris
[2] Dfns

Abstract. We formally define *Threshold Signatures as-a-Service (TSaaS)*, in which the honest parties performing the threshold signature *respond only to the signing requests of a designated client*. This model captures the mainstream industrial use case of threshold signatures which is to implement *Wallets as-a-Service*.
This new model allows for optimizations of existing threshold signature schemes, in particular in the lattice setting. As a particularly relevant case study, we describe a TSaaS variant of the Threshold ML-DSA scheme from [Celi et al., USENIX'26], called ML-DSaaS, which combines the first two rounds into a single message-independent round that can be pre-processed before the message is known. We first describe a simple version of ML-DSaaS in a model where the client is semi-honest. We then upgrade the construction to withstand a possibly corrupt client, by leveraging existence of a coordinating machine which is present in all real-life deployments of TSaaS. This machine, dubbed the Relayer, filters the requests of the client to the parties and centralizes the communications between them. We provide an implementation of our scheme together with experimental benchmarks. The online phase of our scheme is two to three times faster than the one of [Celi et al., USENIX'26].
Our modification carries over unchanged to many similar threshold signature schemes, provided they are used in the TSaaS setting.

1 Introduction

Threshold signatures. Threshold signatures are protocols allowing a group of parties to collectively generate a valid signature, such that a threshold number of them (say, T out of N) are required to cooperate for signing, while the signature remains unforgeable if an adversary controls up to $T-1$ parties. Recently, there has been significant research interest in constructing *lattice-based* threshold signature schemes, motivated by the need for post-quantum security. See Table 1 for a comparison of such schemes. To the best of our knowledge, no existing lattice-based threshold signature scheme simultaneously achieves all

Cette publication est sous la licence creative commons CC BY ND. Les formules et algorithmes sont sous la licence APACHE 2.0.

three of the following desirable properties: (1) reasonable efficiency, (2) a single online round, and (3) compatibility with a standard signature format. By *online* round we refer to a round which can only be processed after the Client decides the message to be signed.

Definitional contribution: TSaaS***.*** *Wallet-as-a-Service* is a widely deployed commercial concept [13, 31, 25, 17] — part of the digital-asset custody market, which was worth over 600 billion dollars in 2024 and is expected to grow rapidly in the coming years [20] — that has not so far been given a precise cryptographic meaning. We take a first step by formalizing *Threshold Signatures as-a-Service* (TSaaS) in Section 2.

In this model, a designated Client needs to quickly produce signatures but *does not store any secret material*, even in cold memory. Instead, the Client acts as the sole authorized initiator, sending authenticated requests to a signing service of N remote parties. The signing key is secret-shared among these parties and never reconstructed in a single location. The service guarantees signature delivery if T of the N parties are honest. In the simplified model $\mathrm{TSaaS}^{\mathsf{sh}}$ (Section 2), the Client is assumed to be semi-honest: the signature remains unforgeable even if the adversary controls up to $T-1$ parties and sees the entire internal state of the Client. As described in below we then relax this assumption in the full deployment model, in Section 4.

Beyond its definitional interest, the TSaaS setting enables concrete optimizations of lattice-based threshold signatures. In particular, it allows us to construct practical lattice-based threshold signatures with a *single online round*: once the message to be signed is known, no interaction between the parties is needed — they just compute their partial signatures and output them. This efficiency goal mirrors the pre-signature paradigm widely adopted in classical threshold schemes (e.g., FROST [24] or threshold ECDSA [9, 23]), but replicating this performance in the post-quantum setting without significant penalties has proven difficult.

Our one-round ML-DSA $\mathrm{TSaaS}^{\mathsf{sh}}$ ***scheme:*** $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$***.*** We adapt the Threshold ML-DSA scheme from [10] (submitted to the NIST Threshold Call [26] under the name Mithril) to the $\mathrm{TSaaS}^{\mathsf{sh}}$ setting. This scheme outputs standard, NIST-compliant ML-DSA signatures (standardized by NIST in 2024) for $N \leq 6$, perfectly fitting industrial requirements. The original protocol has three rounds, including two online rounds; our TSaaS architecture safely compresses these into a single offline pre-processing round and one online signing round. This transformation preserves all original parameters while reducing the final round's computational cost.

The crucial difference compared to existing schemes is that the Client includes a fresh 256-bit *random salt* in each signing request. Since the adversary cannot predict this salt, many consistency checks become unnecessary, and the first two rounds of [10] collapse into a single message-independent preprocessing round. The TSaaS setting is essential: in the usual threshold signature model, there is no designated Client which can choose a common unpredictable salt for all the parties. The resulting signatures are standard, unmodified ML-DSA signatures

Threshold scheme	Online rounds	Signature output is a NIST standard	notes
[1, 6]	1	✓(supports any signature in theory)	[a]
[15, 33]	1	✗(Raccoon)	
[11, 33]	1	✗(other)	
[8] (Ringtail)	1	✗(Raccoon)	
[5] (Quorus)	$\geqslant 10$	✓(ML-DSA)	
[21] (Olingo)	1	✗(Raccoon)	[b]
[10] (Mithril)	2	✓(ML-DSA)	
[7] (Hermine)	1	✗(Raccoon)	[c]
[27] (THED)	2	✓(ML-DSA)	
This work (TSaaS setting only)	1	✓(ML-DSA)	[d]

[a] No concrete instantiation; costs would be prohibitive when applied to ML-DSA.
[b] Only in the sequential setting and under a weaker security model, otherwise there are 3 online rounds. [c] Provides identifiable aborts and proactive security.
[d] Most efficient scheme in the table in terms of communication and computation costs.

Table 1: Comparison of different lattice-based threshold signatures. For conciseness, we only include threshold signatures that either have a single online round, or output standard signatures.

(see Section 3.4). We sketch our $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ (ML-DSA-as-a-Service) below (the full description can be found in Section 3):

- **Key generation.** Identical to [10]: N partial secret keys are created using *replicated secret sharing*, either by a trusted dealer or by a DKG protocol.
- **Pre-processing round.** This single round replaces the first two rounds in [10], and can be executed before the message is known. As in [10], parties compute their nonces $\mathbf{w}_i = \begin{bmatrix}\mathbf{A}\ \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$, but then they immediately reveal them instead of first publishing commitments to them.
- **Client request.** This is where our TSaaS setting comes into play: the Client creates a client request which contains a message to be signed, a set of nonces obtained during the pre-processing round, and a set of T active parties, and a *256-bit random salt*. In the basic presentation (Section 3.1), the Client generates the salt alone. Whereas in the full real-life deployment (Section 4) with a coordinator called the Relayer $\mathcal{R}$, then the Client's salt is rerandomized by a randomness committed by $\mathcal{R}$ in the preprocessing-then-revealed by $\mathcal{R}$ upon forwarding the Client's request.
- **Online signing round.** Upon receiving an authenticated client request, each active party computes the challenge c by hashing the aggregated nonce $\mathbf{w} := \sum_j \mathbf{w}_j$, the message *and the salt*, and outputs its response $\mathbf{z}_i$. As in ML-DSA, parties perform rejection sampling, meaning that they can abort if their response $\mathbf{z}_i$ does not pass certain checks. Contrary to [10], parties do not verify the other parties' nonces, which significantly reduces computation.
- **Signature aggregation.** Once all active parties have computed their responses, they are aggregated into a standard ML-DSA signature $(c, \mathbf{z} = \sum_i \mathbf{z}_i, \mathbf{h})$. The salt is not part of the signature; it is passed to the verifier as the FIPS 204

[16] context string ctx. Verification uses the *unmodified FIPS 204 algorithm*: $\mathsf{ML\text{-}DSA.Verify}(\mathsf{vk}, \mathsf{msg}, \sigma, \mathsf{ctx})$. See Section 3.4 for details.

We prove unforgeability under the same assumptions and with the same bound as [10, Theorem 2]; benchmarks in Section 5 show a $2-3\times$ speedup in online latency. Our approach carries unchanged over similar threshold signature schemes, e.g., [29] or [30], and could also be used to adapt discrete-log-based Fiat-Shamir threshold signatures, though this is less compelling given existing single-online-round solutions like FROST [24].

Remark 1 (Salting in digital signatures). Random salts appear in several non-threshold signature schemes, e.g. RSA-PSS [2, 12], Picnic [28], SPHINCS+ [4] (where it is called a randomizer), and Falcon [19]. Broadly speaking, the role of the salt in such non-threshold schemes is either to allow for a tight security proof, or to prevent various re-signing attacks. In contrast, in our threshold Fiat-Shamir setting, the salt serves a structurally distinct role: it lets parties reveal nonces before the message is known, which is what enables the round collapse. Without it, known attacks against Fiat-Shamir signatures with pre-processing [14, 3, 22] could be possible. At a very high level, these attacks consist in obtaining several sets of pre-processing tokens $(\mathsf{pp}_i)_i$, and then making several signing queries with these sets of tokens and with well-chosen messages. The adversary can then forge a signature on a new message by computing an appropriate linear combination of the obtained signatures. In our construction, the salt prevents these attacks (intuitively, this is because it randomizes the input of the random oracle in an unpredictable way), and allows for a rigorous security proof.

Remark 2 (What happens if the Client is actively corrupt). By definition, a threshold signing protocol *cannot* prevent a corrupt Client from requesting signatures on messages of its choice; this is handled by orthogonal (typically non-cryptographic) security layers such as access control and policy enforcement. It can, however, ensure unforgeability on messages *never queried* by the Client. Our $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ scheme does not achieve this if the Client is *actively* corrupt: it could choose the salt adaptively and potentially mount ROS-style attacks [14, 3, 22] to forge signatures on unqueried messages. Next, we address this issue by upgrading $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ to $\mathsf{ML\text{-}DSaaS}^{r}$.

Full real-life deployment tolerating a corrupt Client: $\mathsf{ML\text{-}DSaaS}^{r}$***.*** We upgrade $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ to tolerate the degraded scenario where the Client is corrupt. To preserve the single online round, we leverage a structural feature of real-world Wallet-as-a-Service deployments [13, 17]: the Client, $\mathcal{C}$, and the signing parties do not communicate directly with each other, but through a Relayer denoted $\mathcal{R}$ (also known as "coordinator") that serves as the system's API gateway. This is the case in most deployments [25, 13], since signing parties typically run in isolated environments (HSMs, TEEs, or private networks) that are not directly reachable by the Client (a mobile app, browser extension, or back-end service) nor from each other. As is the standard practice in existing WaaS deployments, the channels between the Client and the parties are end-to-end

authenticated through the Relayer: in practice, each signing request carries an authentication signature $\sigma_{\mathcal{C}}$ that the parties verify, so that a corrupt Relayer can drop or delay requests but cannot alter them.

We exploit this architecture to ensure salt unpredictability also in the degraded scenario where the Client $\mathcal{C}$ is corrupt. We call the resulting protocol $\mathsf{ML\text{-}DSaaS}^r$; the full construction is given in Section 4. The two modifications compared to $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ are:

(i) The Relayer $\mathcal{R}$ *commits* to a random salt $\mathsf{salt}_{\mathcal{R}}$ during pre-processing. As before, the Client provides a fresh salt in its signing request, which we now denote $\mathsf{salt}_{\mathcal{C}}$. Along with the forwarding of the signing request to the parties, $\mathcal{R}$ reveals $\mathsf{salt}_{\mathcal{R}}$. The salt used by the parties is then defined as the XOR sum $\mathsf{salt} := \mathsf{salt}_{\mathcal{C}} \oplus \mathsf{salt}_{\mathcal{R}}$.

(ii) $\mathcal{R}$ forwards at most one Client request $\mathsf{cr} = (\mathsf{sid}, m, \mathsf{salt}_{\mathcal{C}})$ per (pre-processed) session identifier sid.

Hence, the online signing phase protocol can be summarized by the following diagram, where $\mathcal{R}$ is committed to $\mathsf{salt}_{\mathcal{R}}$ before seeing the $\mathsf{salt}_{\mathcal{C}}$:

$$\mathcal{C} \xrightarrow{\mathsf{cr} = (\mathsf{sid}, \mathsf{salt}_{\mathcal{C}}, \mathsf{msg})} \mathcal{R} \xrightarrow{\mathsf{cr}\ \&\ \text{reveals}\ \mathsf{salt}_R} \mathcal{P}_1, \ldots, \mathcal{P}_N$$

As a result we obtain unforgeability as long as the Client $\mathcal{C}$ and the Relayer $\mathcal{R}$ are not both actively corrupt, essentially because $\mathsf{salt} := \mathsf{salt}_{\mathcal{C}} \oplus \mathsf{salt}_{\mathcal{R}}$ remains unpredictable to the adversary.

Table 2: Unforgeability under different corruption scenarios. We consider at most $T-1$ party corruptions. We write ✓$\mathcal{C}$ (resp. ✗$\mathcal{C}$) for an honest (resp. corrupt) Client and ✓$\mathcal{R}$ (resp. ✗$\mathcal{R}$) for an honest (resp. corrupt) Relayer. In the 2PC row, $\mathcal{R}$ corresponds to the remote server.

			Unforgeability			
Scheme	**Online rounds**	**Client stores secret**	✓$\mathcal{C}$ ✓$\mathcal{R}$	✓$\mathcal{C}$ ✗$\mathcal{R}$	✗$\mathcal{C}$ ✓$\mathcal{R}$	✗$\mathcal{C}$ ✗$\mathcal{R}$
Delegated Mithril [10]	2	No	✓	✓	✓	✓
2PC from [10]	2	Yes	✓	✓	✓	✗
ML-DSaaS^r	**1**	No	✓	✓	✓	✗

Discussion and comparison. Our TSaaS model captures the *Network Hosted Key* (NHK) paradigm, where all key shares are held by remote servers (e.g., Dfns [13]); this contrasts with the *User Hosted Key* (UHK) model (e.g., Fireblocks [17], Coinbase App [25], Zengo [32], Fordefi [18]), where the Client holds an MPC key share. Table 2 compares three paradigms for threshold ML-DSA, assuming at most $T-1$ party corruptions and a Relayer $\mathcal{R}$: (a) Mithril [10] used as a delegated signing service; (b) Mithril [10] in a 2-of-2 UHK deployment between a Client and a single server, denoted "2PC"; and (c) our $\mathsf{ML\text{-}DSaaS}^r$

scheme. Even when the Client is corrupt, $\mathsf{ML\text{-}DSaaS}^r$ retains unforgeability as long as the Relayer is honest, matching the guarantees of 2PC. See Table 2 for a comparison of the guarantees provided by these three paradigms.

2 Formal definitions

In this section, we provide a new syntax and new correctness and unforgeability definitions in order to model Threshold Signatures as-a-Service. We first formalize a simplified model, denoted $\mathrm{TSaaS}^{\mathsf{sh}}$, which assumes a semi-honest Client.

2.1 Syntax

We adapt the syntax of a threshold signature from [10, Section 2.2]. For concreteness, we focus on schemes with one pre-processing round and one online round; the syntax generalizes to more rounds. The significant change compared to usual definitions is the explicit role of the Client.

Definition 1. *A simplified Threshold Signature as-a-Service (*$\mathrm{TSaaS}^{\mathsf{sh}}$*) is comprised of algorithms whose syntax is described below. We denote by* $\mathsf{act} \subseteq [N]$ *a set of* T *parties that take part in signing. Each party* i *maintains a local state* st_i*.*

- $\mathsf{Keygen}(\kappa, N, T) \to (\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]})$*. Takes as input a security parameter* κ*, the total number of parties* N *and the reconstruction threshold* $T \leqslant N$*. Outputs a verification key* vk*, and partial secret keys* $(\mathsf{sk}_i)_{i\in[N]}$*.*
- $\mathsf{PP}(i, \mathsf{st}_i) \to (\mathsf{pp}_i, \mathsf{st}_i)$*. The local pre-processing algorithm takes as input the state* st_i *of party* i*, and outputs a pre-processing token* pp_i *and an updated state* st_i*.*
- $\mathsf{Client}(\mathsf{msg}, \mathsf{act}, (\mathsf{pp}_i)_{i\in\mathsf{act}}) \to \mathsf{cr}$*. The Client creates a signing request* cr *based on a message* msg *to be signed, a set of* T *active parties* act*, and the preprocessing tokens* $(\mathsf{pp}_i)_{i\in\mathsf{act}}$*.*
- $\mathsf{PSign}(\mathsf{vk}, \mathsf{cr}, i, \mathsf{sk}_i, \mathsf{st}_i) \to (\mathsf{sig}_i, \mathsf{st}_i)$*. The partial signing algorithm takes as input the verification key* vk*, a client request* cr*, and the secret key* sk_i *and the state* st_i *of user* i*. It outputs a partial signature* sig_i*, and an updated state* st_i*.*
- $\mathsf{Combine}(\mathsf{vk}, \mathsf{cr}, (\mathsf{sig}_i)_{i\in\mathsf{act}}) \to (\mathsf{sig}, \mathsf{ctx})$*. Takes as input the verification key* vk*, the client request* cr *and the partial signatures output by active parties* $(\mathsf{sig}_i)_{i\in\mathsf{act}}$*. Outputs a final signature* sig *alongside a context string* ctx*.*
- $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}, \mathsf{ctx}) \to 1(\mathsf{accept})/0(\mathsf{reject})$*. Takes as input a verification key* vk*, a message* msg*, a signature* sig *and a context string* ctx*.*

Remark 3. We instantiate Definition 1 for $\mathsf{ML\text{-}DSA}$ in Section 3, where Verify is exactly the standard FIPS 204 [16] verification algorithm. Importantly, FIPS 204 also allows for a *context string* ctx which is signed together with the message. We explicitly include ctx in our syntax because our construction makes essential use of it.

2.2 Correctness

We adapt the correctness definition from [10, Definition A.4] to incorporate the Client; as in [10], correctness (that any produced output is a valid signature) is subsumed into p-termination, where p accounts for aborts due to rejection sampling.

$\mathsf{Game}^{\mathsf{TS-corr}}_{\mathsf{TS}}(\kappa, N, T, \mathsf{msg}, \mathsf{act})$

```
(vk, (sk_i)_{i∈[N]}) ← Keygen(κ, N, T)
for i ∈ [N]
    st_i := ∅
for i ∈ act
    (pp_i, st_i) ← PP(i, st_i)
cr ← Client(msg, act, (pp_i)_{i∈act})
for i ∈ act
    (sig_i, st_i) ← PSign(vk, cr, i, sk_i, st_i)
if ∃i ∈ act, sig_i = ⊥
    return ⊥
(sig, ctx) := Combine(vk, cr, (sig_i)_{i∈act})
return Verify(vk, msg, sig, ctx)
```

Fig. 1: Correctness game for the 1-online round TSaaS scheme TS (with aborts).

Definition 2. *Let* TS *be a* $\mathrm{TSaaS}^{\mathsf{sh}}$ *scheme. For any supported number of parties* N*, threshold* T*, active set* $\mathsf{act} \subseteq [N]$ *of size at least* T*, and message* $\mathsf{msg} \in \{0,1\}^*$*, we say that* TS *p-terminates (where p accounts for aborts during signing) if:*

$$\Pr[\mathsf{Game}^{\mathsf{TS-corr}}_{\mathsf{TS}}(\kappa, N, T, \mathsf{msg}, \mathsf{act}) \neq \bot] \geq p \quad \textit{(liveness)}$$

$$\Pr[\mathsf{Game}^{\mathsf{TS-corr}}_{\mathsf{TS}}(\kappa, N, T, \mathsf{msg}, \mathsf{act}) = 0] = \mathsf{negl}(\kappa) \quad \textit{(correctness)}$$

where $\mathsf{Game}^{\mathsf{TS-corr}}_{\mathsf{TS}}$ *is defined in Figure 1.*

Remark 4. Prop. 1 discusses p-termination for our $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ instantiation. In the full real-life deployment model with a Relayer (Section 4), liveness is not guaranteed if the Relayer is corrupt because it could simply block all the traffic.

2.3 Unforgeability

In this simplified $\mathrm{TSaaS}^{\mathsf{sh}}$ model, and contrary to usual threshold signature definitions, the adversary does not interact with individual honest parties: all queries go through oracles OPP and $\mathsf{OClient}$, which enforce that all honest parties receive the same inputs. In more detail, we adapt the unforgeability definition from [10, Definition A.3] as follows. The adversary corrupts up to $T-1$ parties and receives their secret keys. The adversary can call the $\mathsf{OClient}$ oracle with arbitrary message, set of active parties, and set of pre-processed nonces; the resulting client request is forwarded to the honest parties, but the Client's internal

workings remain outside the adversary's control. The adversary wins if it forges a valid signature on a message not previously queried to $\mathsf{OClient}$. Note that in the full real-life deployment model of Section 4 we will allow again the adversary to interact with honest parties in the unforgeability definition. Specifically, the parties will receive the Client's requests forwarded by a possibly corrupt Relayer.

$\mathsf{Game}^{\mathsf{TS-UF}}_{\mathsf{TS},\mathcal{A}}(\kappa, N, T)$

$Q_{\mathsf{msg}} := \emptyset$; $\mathsf{Sessions} := \emptyset$
$\mathsf{corrupt}, \mathsf{st}_{\mathcal{A}} \leftarrow \mathcal{A}(\kappa, N, T)$
assert $\mathsf{corrupt} \subseteq [N] \wedge |\mathsf{corrupt}| < T$
$\mathsf{honest} := [N] \backslash \mathsf{corrupt}$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i \in [N]}) \leftarrow \mathsf{Keygen}(\kappa, N, T)$
for $i \in \mathsf{honest}$
  $\mathsf{st}_i := \emptyset$
$(\mathsf{msg}, \mathsf{sig}, \mathsf{ctx}) \leftarrow \mathcal{A}^{\mathsf{H},\mathsf{OPP},\mathsf{OClient}}(\mathsf{st}_{\mathcal{A}}, \mathsf{vk}, (\mathsf{sk}_i)_{i \in \mathsf{corrupt}})$
return $(\mathsf{msg} \notin Q_{\mathsf{msg}}) \wedge \mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}, \mathsf{ctx})$

$\mathsf{OPP}(\mathsf{act})$

assert $|\mathsf{act}| = T$
for $i \in \mathsf{act} \cap \mathsf{honest}$
  $(\mathsf{pp}_i, \mathsf{st}_i) \leftarrow \mathsf{PP}(i, \mathsf{st}_i)$
$\mathsf{Sessions} := \mathsf{Sessions} \cup \{(\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}\}$
return $(\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}$

$\mathsf{OClient}(\mathsf{act}, \mathsf{msg}, (\mathsf{pp}_i)_{i \in \mathsf{act}})$

$Q_{\mathsf{msg}} := Q_{\mathsf{msg}} \cup \{\mathsf{msg}\}$
assert $|\mathsf{act}| = T$
assert $(\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}} \in \mathsf{Sessions}$
Update $\mathsf{Sessions} := \mathsf{Sessions} \setminus \{(\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}\}$
$\mathsf{cr} \leftarrow \mathsf{Client}(\mathsf{msg}, \mathsf{act}, (\mathsf{pp}_i)_{i \in \mathsf{act}})$
for $i \in \mathsf{act} \cap \mathsf{honest}$
  $(\mathsf{sig}_i, \mathsf{st}_i) \leftarrow \mathsf{PSign}(\mathsf{vk}, \mathsf{cr}, i, \mathsf{sk}_i, \mathsf{st}_i)$
return $(\mathsf{cr}, (\mathsf{sig}_i)_{i \in \mathsf{act} \cap \mathsf{honest}})$

Fig. 2: Unforgeability game for the TSaaS scheme TS. H denotes oracle access to all the random oracles. If a call to PP or PSign fails (e.g. when rejection sampling aborts), it returns $\perp$; the oracle still returns the responses from the other honest parties.

Definition 3. *Let* TS *be a* $\mathrm{TSaaS}^{\mathsf{sh}}$ *scheme. The advantage of an adversary* $\mathcal{A}$ *against the unforgeability of* TS *is defined as*

$$\mathsf{Adv}^{\mathsf{TS-UF}}_{\mathsf{TS},\mathcal{A}} := \Pr\left[\mathsf{Game}^{\mathsf{TS-UF}}_{\mathsf{TS},\mathcal{A}}(\kappa, N, T) = 1\right] \tag{1}$$

where $\mathsf{Game}^{\mathsf{TS-UF}}_{\mathsf{TS},\mathcal{A}}(\kappa, N, T)$ *is defined in Figure 2. We say that* TS *is unforgeable (in the random oracle model) if for all PPT adversaries* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{TS-UF}}_{\mathsf{TS},\mathcal{A}} \leqslant \mathsf{negl}(\kappa)$.

OPP generates honest preprocessing tokens and stores them in $\mathsf{Sessions}$. Each $\mathsf{OClient}$ query must reference valid, unconsumed preprocessing tokens previously

established during an OPP query; sessions are consumed upon use, preventing the adversary from reusing honest preprocessing states across different signing attempts. One can observe that our definition guarantees unforgeability even if the Client is *passively corrupt*, i.e., even if the adversary can observe its entire internal state, since the Client does not have any secret material.

Use of Sessions. In practice, the Client collects $(\mathsf{pp}_i)_{i\in\mathsf{act}}$ from all parties and later uses them for a signing request. In the unforgeability game, Sessions stores only the *honest* parties' tokens, as the adversary can choose its own tokens adaptively, after seeing the honest ones. Equivalently, a more verbose but perhaps more intuitive way to model this would have been to ask the adversary to send its pre-processing tokens after having seen the honest ones, and then add all the tokens in Sessions.

3 Threshold ML-DSA as-a-Service

In this section, we construct an ML-DSA TSaaS scheme, in the following sense:

Definition 4. *A* TSaaS *scheme is an* ML-DSA TSaaS *scheme (*ML-DSaaS*) if its verification algorithm is identical to the* ML-DSA *verification algorithm (i.e. Algorithm 3 of FIPS 204 [16]).*

3.1 Description of our simplified scheme ML-DSaaS^{sh}

We first present our simplified scheme, ML-DSaaS^{sh}, assuming a semi-honest client and no relayer. It adapts [10] by combining the first two rounds into a single message-independent round without nonce commitments. We use the same notations as [10], and refer to it for the definitions of parameters (recalled in Table 4) and helper algorithms.

Key generation. The key generation is identical to the one from [10]. It assumes a trusted dealer to sample and distribute the shares using replicated secret sharing.

Pre-processing. This algorithm combines the first two rounds from [10]. Each party samples randomness $\mathbf{r}_i$, computes its nonce $\mathbf{w}_i$ and reveals it, without committing to it and independently of the message.

Client request. The Client collects the nonces $(\mathbf{w}_j)_{j\in\mathsf{act}}$ from the parties and builds a request cr with a *256-bit random salt.*

Signing. Each party retrieves $\mathbf{r}_i$ from its state, and reads $(\mathbf{w}_j)_{j\in\mathsf{act}}$ from the request. Without verifying the other parties' nonces (unlike [10]), it computes the challenge c by hashing the aggregated nonce $\mathbf{w}$, the message, and the salt.

Combining and verifying. Combine uses the salt to compute the challenge c and outputs both the standard ML-DSA signature sig and the FIPS 204 context string ctx. Verify uses the unmodified FIPS 204 algorithm taking ctx as input.

$\mathsf{PP}(i, \mathsf{st}_i)$

$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
$\mathbf{w}_i := \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{r}_i$
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathbf{w}_i, \mathbf{r}_i)\}$
return $(\mathsf{pp}_i := \mathbf{w}_i, \mathsf{st}_i)$

$\mathsf{Client}(\mathsf{msg}, \mathsf{act}, (\mathsf{pp}_i)_{i \in \mathsf{act}})$

$\mathsf{salt} \leftarrow \{0,1\}^{256}$
$\mathsf{cr} := (\mathsf{msg}, \mathsf{act}, (\mathsf{pp}_i)_{i \in \mathsf{act}}, \mathsf{salt})$
return cr

$\mathsf{PSign}(\mathsf{vk}, \mathsf{cr}, i, \mathsf{sk}_i, \mathsf{st}_i)$

Parse $(\mathsf{msg}, \mathsf{act}, (\mathbf{w}_j)_{j \in \mathsf{act}}, \mathsf{salt}) := \mathsf{cr}$
assert $i \in \mathsf{act}$ and $\exists \mathbf{r}_i, (\mathbf{w}_i, \mathbf{r}_i) \in \mathsf{st}_i$
$\mathsf{st}_i := \mathsf{st}_i \backslash \{(\mathbf{w}_i, \mathbf{r}_i)\}$
$\mu \in \{0,1\}^{512} := \mathsf{H}\big(\mathsf{tr}||0x00||\mathsf{IntegerToBytes}(|\mathsf{salt}|, 1)||\mathsf{salt}||\mathsf{msg}\big)$
$\mathbf{w} := \sum_{j \in \mathsf{cr.act}} \mathbf{w}_j$
$\mathbf{w}_\top := \mathsf{HighBits}(\mathbf{w}, 2\gamma_2)$
$\tilde{c} \in \{0,1\}^{256} = \mathsf{H}(\mu||\mathbf{w}_\top); c := \mathsf{SampleInBall}(\tilde{c})$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{z}_i \leftarrow \mathsf{HRej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, r, r', \nu, M; \mathbf{r}_i)$
if $\mathbf{z}_i = \perp$ **then abort**
$\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
return $(\mathsf{sig}_i := \mathbf{z}_i^{(1)}, \mathsf{st}_i)$

$\mathsf{Combine}(\mathsf{vk} = (\rho, \mathbf{t}_\top), \mathsf{cr}, (\mathsf{sig}_i)_{i \in \mathsf{cr.act}}) \rightarrow (\mathsf{sig}, \mathsf{ctx})$

Parse $\mathsf{cr.pp} = (\mathbf{w}_i)_i$ and $(\mathsf{sig}_i)_i = (\mathbf{z}_i^{(1)})_i$
$\mathsf{ctx} \leftarrow \mathsf{cr.salt}$
$\mu \in \{0,1\}^{512} := \mathsf{H}\big(tr||0x00||\mathsf{IntegerToBytes}(|\mathsf{ctx}|, 1)||\mathsf{ctx}||\mathsf{cr.msg}\big)$
$\mathbf{w} := \sum_{i \in \mathsf{cr.act}} \mathbf{w}_i; \mathbf{w}_\top := \mathsf{HighBits}(\mathbf{w}, 2\gamma_2)$
$\tilde{c} := \mathsf{H}(\mu||\mathbf{w}_\top)$
$c := \mathsf{SampleInBall}(\tilde{c})$
$\mathbf{z}^{(1)} := \sum_{i \in \mathsf{cr.act}} \mathbf{z}_i^{(1)}$
$\delta := \mathbf{w} - (\mathbf{A}\mathbf{z}^{(1)} - 2^d c \mathbf{t}_\top)$
$\mathbf{h} := \mathsf{MakeHint}_q(\delta, \mathbf{A}\mathbf{z}^{(1)} - 2^d c \mathbf{t}_\top, 2\gamma_2)$
if $||\mathbf{z}^{(1)}||_\infty \geqslant \gamma_1 - \beta$ or $||\delta||_\infty > \gamma_2$ or $||\mathbf{h}||_1 > \omega$
$\llcorner$ **abort**
return $(\mathsf{sig} := (\tilde{c}, \mathbf{z}^{(1)}, \mathbf{h}), \mathsf{ctx})$

$\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}, \mathsf{ctx}) = \mathsf{ML\text{-}DSA.Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}, \mathsf{ctx})$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
Parse $\mathsf{sig} := (\tilde{c}, \mathbf{z}^{(1)}, \mathbf{h})$
$\mu \in \{0,1\}^{512} := \mathsf{H}\big(tr||0x00||\mathsf{IntegerToBytes}(|\mathsf{ctx}|, 1)||\mathsf{ctx}||\mathsf{msg}\big)$
$c := \mathsf{SampleInBall}(\tilde{c})$
$\mathbf{w}'_\top := \mathsf{UseHint}_q(\mathbf{h}, \mathbf{A}\mathbf{z}^{(1)} - 2^d c \mathbf{t}_\top, 2\gamma_2)$
return $||\mathbf{z}^{(1)}||_\infty < \gamma_1 - \beta$ and $\tilde{c} = \mathsf{H}(\mu||\mathbf{w}'_\top)$ and $||\mathbf{h}||_1 \leqslant \omega$

Fig. 3: Description of $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$. Gray lines are identical to [10], of which the notation and parameters are recalled in Section B. Helper algorithms ExpandA, Power2Round, HighBits, SampleInBall, RSSRecover, HRej, MakeHint and UseHint are defined in [10].

3.2 Correctness

Proposition 1. $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ *p-terminates (in the sense of Definition 2) with the same probability p as the threshold* $\mathsf{ML\text{-}DSA}$ *scheme from [10].*

Proof sketch. When all parties are honest and no rejection sampling abort occurs, the secret shares reconstruct the full signing key, so the aggregated response satisfies the same algebraic verification equation as $\mathsf{ML\text{-}DSA.Sign}(\mathsf{sk}, \mathsf{msg}, \mathsf{ctx} = \mathsf{salt})$ and passes the same norm checks. Hence $\mathsf{ML\text{-}DSA.Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}, \mathsf{ctx} = \mathsf{salt})$ accepts. Neither of our two modifications affects correctness: the salt is simply the FIPS 204 context string, which is hashed with the message both during signing and verification; and the removed nonce commitments and cross-verifications are only relevant when some parties misbehave, which does not occur in the correctness game. The p-termination probability is therefore the same as in [10]; the precise bound, together with the required assumptions on the parameters, is stated in Section D.

3.3 Unforgeability

Theorem 1. *Let* $\mathcal{A}$ *be an adversary against the unforgeability (Definition 3) of the* $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ *scheme making at most* Q_s *calls to honest parties (we consider that each call to* $\mathsf{OClient}$ *counts as* $|\mathsf{act} \cap \mathsf{honest}|$ *calls to honest parties) and* Q_{H} *calls to the random oracles* H *and* $\mathsf{SampleInBall}$*. There exist adversaries* $\mathcal{B}_{\mathbf{s}}$ *against* $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{s}}}$*,* $\mathcal{B}_{\mathbf{r}}$ *against* $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{r}}}$*,* $\mathcal{B}_{\mathbf{z}}$ *against* $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{z}}}$*, and* $\mathcal{B}_{\mathsf{UF}}$ *against the unforgeability of* $\mathsf{ML\text{-}DSA}$ *running in time* $\mathscr{T}_{\mathcal{B}_{\mathbf{s}}} \approx \mathscr{T}_{\mathcal{B}_{\mathbf{r}}} \approx \mathscr{T}_{\mathcal{B}_{\mathbf{z}}} \approx \mathscr{T}_{\mathcal{B}_{\mathsf{UF}}} \approx \mathscr{T}_{\mathcal{A}}$ *such that*

$$\begin{aligned}\mathsf{Adv}_{\mathcal{A}} \leqslant 2Q_{\mathsf{H}}Q_s 2^{-256} + Q_{\mathsf{H}}^2 2^{-512} + Q_s \frac{M}{M-1}(\mathsf{Adv}_{\mathcal{B}_{\mathbf{r}}} + \mathsf{Adv}_{\mathcal{B}_{\mathbf{z}}}) + \\ Q_s \frac{M}{M-1} \cdot \frac{2\epsilon}{1-\epsilon} + \left(1 + \frac{\epsilon}{M-1}\right)^{Q_s} (2\mathsf{Adv}_{\mathcal{B}_{\mathbf{s}}} + \mathsf{Adv}_{\mathcal{B}_{\mathsf{UF}}}) + \mathsf{negl}(\kappa)\end{aligned}$$

Note that the advantage of the adversary is slightly smaller than in [10], which allows us to safely use the same parameters as them.

The proof is very similar to the one from [10]. Hence, we only give an overview of the proof, and we refer to Section E for the complete proof. As usual, we define a series of hybrids in order to progressively simulate the honest parties without their signing keys.

Game_1. This is the unforgeability game.

Game_2. Here, our TSaaS setting comes into play. At each pre-processing query, the challenger samples a challenge c and its seed $\tilde{c}$. Then, when the adversary makes a signing query for the corresponding session, the challenger programs $\mathsf{H}(\mu||\mathbf{w}_\top) := \tilde{c}$ and $\mathsf{SampleInBall}(\tilde{c}) := c$. Recall that $\mathbf{w}_\top$ denotes the high bits of the aggregated nonce $\mathbf{w}$, and μ is obtained by hashing the message with the public key and the salt. The random salt ensures that with overwhelming probability, the adversary will not already have queried the random oracle H on

the same input ($\mu||\mathbf{w}_\top$), making this game indistinguishable from the previous one.

Game_3 and Game_4. When the adversary makes a pre-processing query, the challenger still samples $\tilde{c}$ and c at random, and also samples the response $\mathbf{z}_i$ for each honest player i using the rejection sampling algorithm (as would normally be done in the signing round). When the corresponding signing query arrives, the challenger replays these cached values.

Game_5 to Game_7. As in [10, Game 7-9]. During pre-processing queries, the challenger computes the honest $\mathbf{w}_i$'s using the $\mathbf{z}_i$'s it has sampled. In case of a rejected $\mathbf{z}_i$ (i.e. $\mathbf{z}_i = \perp$ because of rejection sampling), $\mathbf{w}_i$ is sampled uniformly.

Game_8 to Game_{11}. As in [10, Game 10-13]. The challenger obtains an ML-DSA public key, and uses it to run the unforgeability game without the secret keys. In particular, the honest $\mathbf{z}_i$'s are sampled from an ideal distribution which is independent from the secret keys.

Conclusion. As in [10], we use a successful adversary against Game_{11} to forge an ML-DSA signature.

3.4 FIPS 204 Compliance

The protocol outputs signatures strictly compliant with the NIST standard. FIPS 204 defines the message representative as:

$$\mu = H(\mathsf{tr} \parallel 0x00 \parallel \mathsf{IntegerToBytes}(|\mathsf{ctx}|, 1) \parallel \mathsf{ctx} \parallel \mathsf{msg})$$

where ctx is an application-supplied context string of at most 255 bytes. Our 256-bit (32-byte) salt fits within this limit. Hence, the Client simply passes the salt as the context string. The Combine algorithm (Figure 3) explicitly outputs $\mathsf{ctx} \leftarrow \mathsf{salt}$. To verify, one calls the *unmodified* FIPS 204 verification algorithm:

$$\mathsf{ML\text{-}DSA.Verify}(\mathsf{vk}, \mathsf{msg}, \sigma, \mathsf{ctx} = \mathsf{salt})$$

The verifier must know the salt value, but this is not different from any FIPS 204 deployment with non-empty context string. In practice, the salt is transmitted alongside the message as application-level metadata (e.g., in a transaction header), exactly as any other context string would be.

Remark 5 (Embedding the salt in the message). If the application-level context string ctx together with the 32-byte salt would exceed the 255-byte FIPS 204 limit, the Client can instead embed the salt into the message: parties sign $\mathsf{msg}' = \mathsf{salt}||\mathsf{msg}$, and the verifier calls the standard $\mathsf{ML\text{-}DSA.Verify}(\mathsf{vk}, \mathsf{msg}', \mathsf{sig}, \mathsf{ctx})$. Correctness now holds with respect to msg' rather than msg. Security is unaffected, since the salt adds the same 256 bits of entropy to μ regardless of its position in the hash input. A detailed discussion is given in Section C.

4 Full real-life deployment tolerating a corrupt Client

In this section we specify and analyze the $\mathsf{ML\text{-}DSaaS}^r$ protocol, which is an extension of $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$, which now tolerates a corrupt Client $\mathcal{C}$. It preserves the

targeted properties of $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$: it has a single online round, outputs standard $\mathsf{ML\text{-}DSA}$ signatures, and remains highly efficient. In order to preserve the single online round, we leverage a feature of existing Wallet-as-a-Service deployments [13, 17]: the communication network is star-shaped, centered around a machine called the Relayer, denoted $\mathcal{R}$. All end-to-end channels between parties remain authenticated, e.g., via signatures, but go through $\mathcal{R}$. Hence, when an honest party $\mathcal{P}_i$ sends a message m to all parties via $\mathcal{R}$, dubbed a *multicast*, then a corrupt $\mathcal{R}$ can possibly refuse to deliver m to some or all parties, but not alter the content of the message delivered. Note that a corrupt sender $\mathcal{P}_i$ could send conflicting messages to different parties.

Note that it would be straightforward to further generalize the specifications of $\mathsf{ML\text{-}DSaaS}^{r}$ into an upgrade, TSaaS^{r}, of the model $\mathrm{TSaaS}^{\mathsf{sh}}$ (essentially by abstracting-out the commitment and salt of the Relayer). We refrained to, for conciseness and clarity.

4.1 Protocol $\mathsf{ML\text{-}DSaaS}^{r}$

The protocol has an offline phase called Π_{PreSign}, and an online phase called Π_{Sign}, both described in Figure 4. They are bound by a session identifier, sid, chosen by the Relayer $\mathcal{R}$. We first give a high-level description.

1. **Pre-signatures generation (Π_{PreSign}).** $\mathcal{R}$ samples a salt $\mathsf{salt}_{\mathcal{R}}$, stores $(\mathsf{sid}, \mathsf{salt}_{\mathcal{R}})$ in its table of pending salts, $\mathsf{Salts}_{\mathcal{R}}$, generates the commitment $\mathsf{com}_{\mathcal{R}} = \mathsf{H}_{\mathcal{R}}(\mathsf{salt}_{\mathcal{R}})$ via a random oracle $\mathsf{H}_{\mathcal{R}}$, and then multicasts it. Upon receiving the commitment, each party $\mathcal{P}_i$ runs the pre-processing algorithm PP to generate its nonce $\mathbf{w}_i$ and multicasts it. Parties then run the Unanimous-or-Abort sub-protocol (UoA, Figure 5) to check unanimity of the transcript $(\mathsf{sid}, \mathsf{act}, \mathsf{com}_{\mathcal{R}}, (\mathbf{w}_j)_j)$. On acceptance, each party binds the transcript to its local state; otherwise it aborts.
2. **Signing (Π_{Sign}).** The Client $\mathcal{C}$ samples $\mathsf{salt}_{\mathcal{C}}$, signs $\sigma_{\mathcal{C}} \leftarrow \mathsf{Sign}_{\mathcal{C}}(\mathsf{sid} \| \mathsf{salt}_{\mathcal{C}} \| H(\mathsf{msg}))$, and sends $\mathsf{cr} = (\mathsf{sid}, \mathsf{msg}, \mathsf{salt}_{\mathcal{C}}, \sigma_{\mathcal{C}})$ to the Relayer $\mathcal{R}$. The Relayer checks that no prior request was done for this sid, then multicasts $(\mathsf{cr}, \mathsf{salt}_{\mathcal{R}})$, opening its commitment. Each party checks the opening, i.e., that $\mathsf{H}_{\mathcal{R}}(\mathsf{salt}_{\mathcal{R}}) = \mathsf{com}_{\mathcal{R}}$, and checks the signature of $\mathcal{C}$ on cr. Then it re-randomizes the salt provided by $\mathcal{C}$ by XORing it with $\mathsf{salt}_{\mathcal{R}}$, i.e., $\mathsf{salt} \leftarrow \mathsf{cr}.\mathsf{salt}_{\mathcal{C}} \oplus \mathsf{salt}_{\mathcal{R}}$, and produces its partial signature using salt with the same PSign as before.

The check by $\mathcal{R}$, of freshness of the sid, is formalized by $\mathsf{AcceptClientRequest}$ below. It is the rampart against the adversary which would observe $\mathcal{R}$ releasing its $\mathsf{salt}_{\mathcal{R}}$, then would make the corrupt $\mathcal{C}$ send many other signing requests to $\mathcal{R}$ for the same sid. Indeed, since the adversary would then know the corresponding $\mathsf{salt}_{\mathcal{R}}$, it could potentially mount a ROS [3]-like attack –at least our unforgeability proof would not go through. The detailed description of $\mathsf{ML\text{-}DSaaS}^{r}$ makes use of helper algorithms, of which $\mathsf{Combine}$ and Verify which remain unchanged. We now describe the new or modified helper algorithms:

- $\mathsf{Keygen}(\kappa, N, T) \rightarrow (\mathsf{vk}, (\mathsf{sk}_i)_{i \in [N]})$. Identical to the simplified model, with the addition that the Client generates an authentication key pair $(\mathsf{pk}_{\mathcal{C}}, \mathsf{sk}_{\mathcal{C}}) \leftarrow \mathsf{SigGen}(1^{\kappa})$ and registers $\mathsf{pk}_{\mathcal{C}}$ with all parties before the first session.

- $\mathsf{Client}(\mathsf{msg}, \mathsf{sid}) \to \mathsf{cr}$. The Client samples $\mathsf{salt}_{\mathcal{C}} \leftarrow \{0,1\}^{256}$, computes $\sigma_{\mathcal{C}} \leftarrow \mathsf{Sign}_{\mathcal{C}}(\mathsf{sid} \| \mathsf{salt}_{\mathcal{C}} \| \mathsf{H}(\mathsf{msg}))$, and returns $\mathsf{cr} := (\mathsf{sid}, \mathsf{msg}, \mathsf{salt}_{\mathcal{C}}, \sigma_{\mathcal{C}})$.
- $\mathsf{VerifyClientSignature}(\mathsf{cr}) \to 1(\mathsf{accept})/0(\mathsf{reject})$. Verifies the Client's authentication signature on cr, i.e., outputs $\mathsf{Verify}(\mathsf{pk}_{\mathcal{C}}, (\mathsf{cr.sid} \| \mathsf{cr.salt}_{\mathcal{C}} \| \mathsf{H}(\mathsf{cr.msg})), \mathsf{cr}.\sigma_{\mathcal{C}})$.
- $\mathsf{AcceptClientRequest}(\mathsf{cr}, \mathsf{Salts}_{\mathcal{R}}) \to 1(\mathsf{accept})/0(\mathsf{reject})$. Run by the Relayer $\mathcal{R}$, it asserts that there exists an entry of the form $(\mathsf{cr.sid}, \mathsf{act}, \mathsf{salt}_{\mathcal{R}}) \in \mathsf{Salts}_{\mathcal{R}}$, i.e., that the session exists and has not been consumed. On acceptance, removes the entry $(\mathsf{cr.sid}, \mathsf{act}, \mathsf{salt}_{\mathcal{R}})$ from $\mathsf{Salts}_{\mathcal{R}}$.
- $\mathsf{UoA}(\mathsf{act})$ (Unanimous-or-Abort). It is the only *interactive sub-protocol* of this list, and is analogous to the "weak broadcast" of [9, §3.5.1]. Each party computes a digest of the full session transcript which it observed and multicasts it (through the authenticated channels via $\mathcal{R}$). A party accepts a transcript only if it received it (its digest) from *all* parties, else, it aborts. Hence a corrupt $\mathcal{R}$ can cause aborts but not acceptance of conflicting transcripts. UoA is further formalized in Appendix A.

Π_{PreSign}

Relayer $\mathcal{R}$:
1: Choose a 256-bit sid not used before and a T-sized subset $\mathsf{act} \subset [n]$
2: sample $\mathsf{salt}_{\mathcal{R}} \leftarrow \{0,1\}^{256}$ ▷ future salt randomizer
3: set $\mathsf{com}_{\mathcal{R}} := \mathsf{H}_{\mathcal{R}}(\mathsf{salt}_{\mathcal{R}})$ ▷ commit to randomizer
4: store $(\mathsf{sid}, \mathsf{act}, \mathsf{salt}_{\mathcal{R}})$ in $\mathsf{Salts}_{\mathcal{R}}$
5: Multicast $(\mathsf{sid}, \mathsf{act}, \mathsf{com}_{\mathcal{R}})$ to parties in act and to $\mathcal{C}$

Each Party $\mathcal{P}_i$:
6: **upon** receiving $(\mathsf{sid}, \mathsf{act}, \mathsf{com}_{\mathcal{R}})$ from $\mathcal{R}$:
7: **assert:** $i \in \mathsf{act}$.
8: **assert:** $(\mathsf{sid}, _, _, _, _) \notin \mathsf{st}_i$.
9: Run $(\mathbf{w}_i, \mathbf{r}_i) \leftarrow \mathsf{PP}^{(\dagger)}(i, \mathsf{st}_i)$
10: Multicast $(\mathsf{sid}, \mathbf{w}_i)$ to parties in act
11: **upon** receiving $(\mathsf{sid}, (\mathbf{w}_j)_{j \in \mathsf{act} \setminus \{i\}})$:
12: Run $\mathsf{UoA}(\mathsf{act})$ on $\big(\mathsf{sid}, \mathsf{act}, \mathsf{com}_{\mathcal{R}}, (\mathbf{w}_j)_{j \in \mathsf{act}}\big)$
13: **if** UoA returns accept:
14: $\mathsf{st}_i \leftarrow \mathsf{st}_i \cup \{(\mathsf{sid}, \mathsf{act}, (\mathbf{w}_j)_{j \in \mathsf{act}}, r_i, \mathsf{com}_{\mathcal{R}})\}$
15: **else** discard session sid and abort

(†) obvious adaptation of Figure 3 where st_i is not updated and $\mathbf{r}_i$ is explicitly returned

$\Pi_{\mathsf{Sign}}(\mathsf{sid}, \mathsf{msg})$

Client $\mathcal{C}$:
1: $\mathsf{cr} = (\mathsf{sid}, \mathsf{salt}_{\mathcal{C}}, \mathsf{msg}, \sigma_{\mathcal{C}}) \leftarrow \mathsf{Client}(\mathsf{msg}, \mathsf{sid})$
2: Send cr to the Relayer $\mathcal{R}$.

Relayer $\mathcal{R}$:
3: **assert** $\mathsf{AcceptClientRequest}(\mathsf{cr}, \mathsf{Salts}_{\mathcal{R}}) = 1$
4: Retrieve and remove $(\mathsf{sid}, \mathsf{salt}_{\mathcal{R}})$ from $\mathsf{Salts}_{\mathcal{R}}$
5: Multicast $(\mathsf{cr}, \mathsf{salt}_{\mathcal{R}})$ to the parties in act

Each Party $\mathcal{P}_i$:
6: **upon** receiving $(\mathsf{cr}, \mathsf{salt}_{\mathcal{R}})$ from $\mathcal{R}$:
7: **assert** $\exists\, (\mathsf{cr.sid}, \mathsf{act}, (\mathbf{w}_j)_{j \in \mathsf{act}}, \mathbf{r}_i, \mathsf{com}_{\mathcal{R}}) \in \mathsf{st}_i$ then remove it from st_i
8: **assert** $i \in \mathsf{act}$
9: **assert** $\mathsf{VerifyClientSignature}(\mathsf{cr}) = 1$
10: **assert** $\mathsf{H}_{\mathcal{R}}(\mathsf{salt}_{\mathcal{R}}) = \mathsf{com}_{\mathcal{R}}$
11: Computes $\mathsf{salt} \leftarrow \mathsf{cr.salt}_{\mathcal{C}} \oplus \mathsf{salt}_{\mathcal{R}}$
12: $\widetilde{\mathsf{cr}}$ (‡) $\leftarrow (\mathsf{msg}, \mathsf{act}, (\mathbf{w}_j)_{j \in \mathsf{act}}, \mathsf{salt})$
13: Run $(\mathsf{sig}_i, \mathsf{st}_i) \leftarrow \mathsf{PSign}^{(\dagger)}(\mathsf{vk}, \widetilde{\mathsf{cr}}, i, \mathsf{sk}_i, \mathbf{r}_i)$
14: **return** sig_i to $\mathcal{R}$

(‡) recomputes to itself a client request in the $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ format

(†) obvious adaptation of Figure 3 where $\mathbf{r}_i$ is an explicit input instead of being in the state

Fig. 4: The $\mathsf{ML\text{-}DSaaS}^r$ protocol with a Relayer $\mathcal{R}$.

4.2 Analysis

Remark 6 (Liveness). A corrupt Relayer can block messages, causing the protocol to abort. Liveness therefore requires an honest Relayer, but this is the case in all existing deployments using a Relayer.

We now show that $\mathsf{ML\text{-}DSaaS}^r$ is unforgeable if either the Client or the Relayer is honest (see Table 2 for a comparison of this guarantee with existing constructions). To avoid formally redefining the security definition, we choose to keep this section slightly informal. **We define unforgeability of $\mathsf{ML\text{-}DSaaS}^r$ as follows.** At the start of the game, the adversary $\mathcal{A}$ additionally chooses to corrupt either the Client, $\mathcal{C}$, or the Relayer, $\mathcal{R}$. Throughout the game, $\mathcal{A}$ can make corrupt parties act arbitrarily, while the game simulates the remaining honest machines (including $\mathcal{C}$ and/or $\mathcal{R}$) acting honestly.

- *If $\mathcal{R}$ is honest*, $\mathcal{A}$ has oracle access to $\mathcal{R}$ initiating pre-processings via $\Pi_{\mathsf{PreSign}}(\mathsf{act})$.
- *If $\mathcal{C}$ is honest*, $\mathcal{A}$ has oracle access to $\mathcal{C}$ initiating signing queries via $\Pi_{\mathsf{Sign}}(\mathsf{sid}, \mathsf{msg})$.

Definition of a valid forgery: The adversary wins the unforgeability game if it outputs a valid signature σ over a message msg such that no honest party ever issued a partial signature on msg for any salt. Namely, no honest party ever does the action of running line 13 of Figure 4 (Π_{Sign}), i.e., of running $\mathsf{PSign}(\mathsf{vk}, \widetilde{\mathsf{cr}}, i, \mathsf{sk}_i, \mathbf{r}_i)$ with $\widetilde{\mathsf{cr}}.\mathsf{msg} = \mathsf{msg}$. Note that protocol Π_{Sign} could be straightforwardly enriched with parties further asserting possible external policies, before accepting to issue a partial signature on a $\mathsf{cr.msg}$. Hence, our definition of unforgeability would capture such policies. Defining them is left out of the scope of this paper.

Theorem 2. *Let $\mathcal{A}$ be an adversary against the unforgeability of $\mathsf{ML\text{-}DSaaS}^r$, making at most Q_s calls to honest parties and Q_H calls to the random oracles (including $\mathsf{H}_{\mathcal{R}}$). There exists an adversary $\mathcal{B}$ against the unforgeability of $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ (thus $\mathsf{Adv}_{\mathcal{B}}^{\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}}$ is bounded in Theorem 1), which runs in approximately the same time as $\mathcal{A}$ ($\mathscr{T}_{\mathcal{B}} \approx \mathscr{T}_{\mathcal{A}}$), such that*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{ML\text{-}DSaaS}^r} \leq \mathsf{Adv}_{\mathcal{B}}^{\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}} + Q_s \cdot Q_H / 2^{256}$$

Proof. We start by analyzing the unforgeability game of $\mathsf{ML\text{-}DSaaS}^r$, denoted Game 1. Then we modify it into Game $1'$ in a way that the view of $\mathcal{A}$, hence its advantage, is (almost) preserved. Then we conclude by building a reduction $\mathcal{B}$, i.e., an attacker of the unforgeability of $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$, which uses an adversary $\mathcal{A}$ of Game $1'$ as a sub-routine.

Claim ***(Consistency).*** By construction of UoA, any two honest parties that accept a session hold the same transcript $(\mathsf{sid}, \mathsf{act}, \mathsf{com}_{\mathcal{R}}, (\mathbf{w}_j)_{j \in \mathsf{act}})$. A corrupt $\mathcal{R}$ can prevent some parties from accepting, but cannot break consistency among those that do. In particular, this lets us refer unambiguously to *the* aggregated nonce $\mathbf{w} = \sum_{j \in \mathsf{act}} \mathbf{w}_j$ and *the* commitment $\mathsf{com}_{\mathcal{R}}$ per session.

Game $1'$ **(Deferred $\mathsf{salt}_{\mathcal{R}}$).** If $\mathcal{R}$ is corrupt, then this game is identical to Game 1. *Else, if $\mathcal{R}$ is honest*, we replace $\mathsf{com}_{\mathcal{R}} = \mathsf{H}_{\mathcal{R}}(\mathsf{salt}_{\mathcal{R}})$ with a uniform value and defer the sampling of $\mathsf{salt}_{\mathcal{R}}$ until $\mathcal{R}$ reveals it, at which point we program $\mathsf{H}_{\mathcal{R}}(\mathsf{salt}_{\mathcal{R}})$ on $\mathsf{com}_{\mathcal{R}}$ (unless $\mathsf{salt}_{\mathcal{R}}$ was queried before, in which case it crashes). **Indistinguishability from** Game 1 follows from the fact that $\mathsf{H}_{\mathcal{R}}$ is a random oracle, so $\mathsf{com}_{\mathcal{R}}$ is independent of $\mathsf{salt}_{\mathcal{R}}$; the adversary hits the given $\mathsf{salt}_{\mathcal{R}}$ with probability at most $Q_H/2^{256}$, hence it hits one of the $\mathsf{salt}_{\mathcal{R}}$'s with probability at most $Q_s \cdot Q_H/2^{256}$.

Constructing the adversary $\mathcal{B}$. We describe an adversary $\mathcal{B}$ against the $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ unforgeability game, in particular having access to OPP and $\mathsf{OClient}$, and using as a subroutine an adversary against $\mathsf{Game}\ 1'$, denoted $\mathcal{A}$.

- During Π_{PreSign}. $\mathcal{B}$ calls OPP to obtain $(\mathbf{w}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}$. If $\mathcal{R}$ is corrupt, the adversary provides $\mathsf{com}_{\mathcal{R}}$ directly. If $\mathcal{R}$ is honest, then $\mathcal{B}$ samples $\mathsf{com}_{\mathcal{R}}$ uniformly at random. Once $\mathcal{B}$ obtains the nonces $(\mathbf{w}_i)_{i \in \mathsf{act} \cap \mathsf{corrupt}}$ from $\mathcal{A}$, it simulates the UoA sub-protocol from the transcript $(\mathsf{sid}, \mathsf{act}, \mathsf{com}_{\mathcal{R}}, (\mathbf{w}_j)_{j \in \mathsf{act}})$.
- During Π_{Sign}. Upon $\mathcal{R}$ (corrupt or honest simulated) relaying a valid request from the Client $\mathcal{C}$, $\mathsf{cr} = (\mathsf{sid}, \mathsf{salt}_{\mathcal{C}}, \mathsf{msg}, \sigma_{\mathcal{C}})$ [3] along with a $\mathsf{salt}_{\mathcal{R}}$ which opens its commitment, then $\mathcal{B}$ calls $\mathsf{OClient}(\mathsf{msg}, (\mathbf{w}_j)_{j \in \mathsf{act}})$, where the $(\mathbf{w}_j)_{j \in \mathsf{act}}$ are the common output of the preprocessing of sid. $\mathsf{OClient}$ internally generates a client request $\widetilde{\mathsf{cr}}$, which contains a salt sampled uniformly at random. Then it runs PSign for each honest party and returns to $\mathcal{B}$ the partial signatures of the parties $i \in \mathsf{act} \cap \mathsf{honest}$ generated as $(\mathsf{sig}_i, _) \leftarrow \mathsf{PSign}(\mathsf{vk}, \widetilde{\mathsf{cr}}, i, \mathsf{sk}_i, \mathsf{st}_i)$, together with the $\widetilde{\mathsf{cr}}$.
  - *If $\mathcal{R}$ is honest*, $\mathcal{B}$ sets $\mathsf{salt}_{\mathcal{R}} := \mathsf{salt}_{\mathcal{C}} \oplus \mathsf{salt}$. It varies uniformly independently from the view of $\mathcal{A}$ so far, since salt also does. Hence the $\mathsf{salt}_{\mathcal{R}}$ is consistent with $\mathsf{Game}\ 1'$.
  - *Else if $\mathcal{C}$ is honest*, upon obtaining $\mathsf{salt}_{\mathcal{R}}$ from $\mathcal{R}$, then $\mathcal{B}$ sets $\mathsf{salt}_{\mathcal{C}} := \mathsf{salt}_{\mathcal{R}} \oplus \mathsf{salt}$. It varies uniformly independently from the view of $\mathcal{A}$ so far, as in $\mathsf{Game}\ 1'$, since salt also does.

  Finally, $\mathcal{B}$ simulates the honest parties returning their partial signatures $(\mathsf{sig}_i)_{\mathsf{act} \cap \mathsf{honest}}$. Since they are computed from $\widetilde{\mathsf{cr}}$, which matches what parties recompute in $\mathsf{PSign}(\mathsf{sid}, \mathsf{msg})$, it follows that the $(\mathsf{sig}_i)_{\mathsf{act} \cap \mathsf{honest}}$ are distributed as in $\mathsf{Game}\ 1'$.
- When $\mathcal{A}$ outputs a forgery $(\mathsf{msg}, \mathsf{sig}, \mathsf{ctx})$, $\mathcal{B}$ outputs the same forgery. If $\mathcal{A}$ wins, then by definition no party ever accepted to apply PSign to a signing request for msg from the Client (corrupt or honest simulated). Hence, msg was never queried by $\mathcal{B}$ to its $\mathsf{OClient}$. As a result, $(\mathsf{msg}, \mathsf{sig}, \mathsf{ctx})$ is also a winning forgery for $\mathcal{B}$.

5 Implementation and evaluation

We evaluate the semi-honest variant $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ against the state-of-the-art Threshold $\mathsf{ML\text{-}DSA}$ scheme from [10]; the upgrade to $\mathsf{ML\text{-}DSaaS}^{r}$ adds only a single XOR and one hash verification per party in the online phase, so its impact on the benchmarks is negligible. Both schemes use the same parameters, corresponding to $\mathsf{ML\text{-}DSA\text{-}44}$ (NIST category 2). We focus on the *online phase*, since it is the critical path latency experienced by a client. Our `Golang` prototype[4] extends the reference implementation of [10] with a libp2p-pubsub (GossipSub)

[3] "Valid" means which passes $\mathsf{AcceptClientRequest}$, i.e., with a sid not consumed yet, and which passes $\mathsf{VerifyClientSignature}$.

[4] Our implementation is available at `https://github.com/Antoineurban/threshold-ml-dsa`

networking stack in place of direct TCP streams (since it requires each party to maintain a dedicated connection to every other party). This introduces serialization overhead compared to direct TCP channels; consequently, our bandwidth figures are *a conservative upper bound on communication costs.*

Experimental Setup. We deployed the protocol on Amazon AWS using `t3.small` instances (2 vCPUs, 2 GiB RAM). To rigorously test the protocol under realistic wide-area network (WAN) conditions, we distributed T active instances across three geographically distant regions: `ap-southeast-2` (Sydney), `eu-central-1` (Frankfurt), and `eu-west-3` (Paris), enforcing a ≥ 290 ms RTT floor by requiring at least one active party in Sydney and one in Europe.

(T, N)	**Online running time [10]**	**Online running time (our work)**	**Online bandwidth [10]**	**Online bandwidth (our work)**
$(2, 3)$	771.48	315.38	33.114	12.989
$(2, 4)$	818.09	362.41	38.138	16.237
$(3, 4)$	1298.00	534.11	90.852	39.898
$(3, 5)$	1584.37	548.52	195.101	85.064
$(4, 5)$	2293.14	936.85	448.522	195.204
$(3, 6)$	1829.88	801.26	266.154	116.861
$(4, 6)$	2365.03	796.96	673.306	295.607
$(5, 6)$	3014.45	1080.00	1189.762	522.345

Table 3: Average running times (in ms) and bandwidths (in kilobytes) of the online parts of the Threshold ML-DSA scheme from [10] and our ML-DSaaS^{sh} version, for different values of N and T.

Methodology and Metrics. We measure the latency and bandwidth of the *Online Phase*, defined as the operations that must block until the message is available: the two last signing rounds and the combining round for [10], versus the single signing round and the combining round for our scheme. Running times of key generation, preprocessing, and verification are essentially the same in both schemes and are omitted. Our metrics are averages that include potential aborts from rejection sampling, unlike [10] which reports averages over successful runs only. This captures the true amortized cost of generating a valid signature.

Results. Table 3 compares the online performance. Our scheme completes in 300–1100 ms, achieving a 2–3× speedup and a comparable reduction in bandwidth over [10]. For the common $(N = 3, T = 2)$ configuration, we achieve an online latency of 315 ms against a 290 ms RTT floor, implying a minimal software overhead. The sharp increase for higher thresholds is driven by the parallel repetition parameter K, which scales superlinearly.

Bibliography

[1] Agrawal, S., Stehlé, D., Yadav, A.: Round-optimal lattice-based threshold signatures, revisited. In: ICALP (2022)

[2] Bellare, M., Rogaway, P.: The exact security of digital signatures-how to sign with RSA and rabin. In: EUROCRYPT (1996)

[3] Benhamouda, F., Lepoint, T., Loss, J., Orrù, M., Raykova, M.: On the (in)security of ROS. J. Cryptology (2022)

[4] Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The SPHINCS+ signature framework. In: CCS (2019)

[5] Bienstock, A., de Castro, L., Escudero, D., Polychroniadou, A., Takahashi, A.: Quorus: Efficient, scalable threshold ML-DSA signatures from MPC. In: Usenix Security (2026)

[6] Boneh, D., Gennaro, R., Goldfeder, S., Jain, A., Kim, S., Rasmussen, P.M.R., Sahai, A.: Threshold cryptosystems from threshold fully homomorphic encryption. In: CRYPTO (2018)

[7] Borin, G., Celi, S., del Pino, R., Espitau, T., Katsumata, S., Niot, G., Prest, T., Takemure, K.: Hermine: An efficient lattice-based FROST-like threshold signature. Cryptology ePrint Archive, Paper 2026/419 (2026)

[8] Boschini, C., Kaviani, D., Lai, R.W.F., Malavolta, G., Takahashi, A., Tibouchi, M.: Ringtail: Practical two-round threshold signatures from learning with errors. In: IEEE SP (2025)

[9] Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N., Peled, U.: UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In: CCS (2020)

[10] Celi, S., del Pino, R., Espitau, T., Niot, G., Prest, T.: Efficient threshold ML-DSA. In: Usenix Security (2026)

[11] Chairattana-Apirom, R., Tessaro, S., Zhu, C.: Partially non-interactive two-round lattice-based threshold signatures. In: ASIACRYPT (2025)

[12] Coron, J.S.: Optimal security proofs for pss and other signature schemes. In: Knudsen, L.R. (ed.) EUROCRYPT (2002)

[13] Dfns: Delegated signing. `https://docs.dfns.co/advanced/delegated-signing`, accessed: 2025-11-24

[14] Drijvers, M., Edalatnejad, K., Ford, B., Kiltz, E., Loss, J., Neven, G., Stepanovs, I.: On the security of two-round multi-signatures. In: IEEE SP (2019)

[15] Espitau, T., Katsumata, S., Takemure, K.: Two-round threshold signature from algebraic one-more learning with errors. J. Cryptol. (2025)

[16] Module-lattice-based digital signature standard. National Institute of Standards and Technology. `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf` (2024)

[17] Fireblocks: Fireblocks key features & capabilities. `https://developers.fireblocks.com/docs/capabilities`, accessed: 2025-11-24

[18] Fordefi: Fordefi wallet as a service. https://docs.fordefi.com/waas/welcome, accessed: 2026-03-24

[19] Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: Falcon: Fast-fourier lattice-based compact signatures over NTRU. https://falcon-sign.info/ (2017), submitted to the NIST Post-Quantum Cryptography Project on November 30, 2017. Accessed: April 13, 2026

[20] Grand View Research: Digital asset custody market (2025–2033): (2025), report ID: GVR-4-68040-770-2

[21] Gur, K.D., Hough, P., Katz, J., Sandsbraten, C., Silde, T.: Olingo: Threshold lattice signatures with DKG and identifiable abort. ePrint 2025/1789 (2025)

[22] Katsumata, S., Lai, Y.F., Reichle, M.: Breaking parallel ros: Implication for isogeny and lattice-based blind signatures. In: PKC (2024)

[23] Katz, J., Urban, A.: Honest-majority threshold ECDSA with batch generation of key-independent presignatures. IACR CIC (2025)

[24] Komlo, C., Goldberg, I.: FROST: Flexible round-optimized schnorr threshold signatures. In: SAC (2021)

[25] Lindell, Y.: Digital asset management with mpc (whitepaper). https://www.coinbase.com/blog/digital-asset-management-with-mpc-whitepaper (2023), coinbase blog post. Accessed: 2025-11-24

[26] National Institute of Standards and Technology (NIST): Threshold cryptography. Computer Security Resource Center (CSRC), https://csrc.nist.gov/projects/threshold-cryptography (2025), last reviewed: April 1, 2025. Accessed: April 13, 2026

[27] Park, J.H., Passelègue, A., Stehlé, D.: THED: Threshold dilithium from FHE. Cryptology ePrint Archive, Paper 2026/638 (2026)

[28] Picnic Team: Picnic: A family of post-quantum secure digital signature algorithms. https://microsoft.github.io/Picnic/ (2026), accessed: 2026-04-22

[29] del Pino, R., Katsumata, S., Maller, M., Mouhartem, F., Prest, T., Saarinen, M.J.: Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In: EUROCRYPT (2024)

[30] del Pino, R., Niot, G.: Finally! a compact lattice-based threshold signature. In: Jager, T., Pan, J. (eds.) PKC (2025)

[31] Shlomovits, O.: Explanation of threshold signatures. https://www.binance.com/en/square/post/17681517589057 (2024), binance blog post. Accessed: 2025-11-24

[32] Zengo: Zengo crypto wallet app. https://zengo.com/crypto-wallet/, accessed: 2026-03-24

[33] Zhu, C., Tessaro, S.: The algebraic one-more misis problem and applications to threshold signatures. In: CRYPTO (2025)

A Unanimous-or-Abort

Figure 5 defines the Unanimous-or-Abort (UoA) sub-protocol executed by the parties at the end of preprocessing: it guarantees that any two accepting parties hold the same session transcript, even if the relayer is corrupt.

UoA(act)

Each Party $\mathcal{P}_i \in \mathsf{act}$**, on input** $\boldsymbol{x_i}$**:**
Sends $d_i := \mathsf{H}(\boldsymbol{x_i})$ to all other parties $\mathcal{P}_j \in \mathsf{act}$ via the end-to-end authenticated channels (routed through $\mathcal{R}$).
Each Party $\mathcal{P}_i \in \mathsf{act}$**:**
if received $d_j == d_i$ for all $j \in \mathsf{act}$
  return accept
else
  return abort

Fig. 5: Unanimous-or-Abort. We abstract the authentication by assuming end-to-end authenticated channels between all parties. A corrupt relayer can cause aborts (losing liveness) but cannot spoof digests (correctness is preserved).

B Parameters

In Table 4, we detail the parameters used in ML-DSaaS following [10].

$\mathscr{R}_q$	Polynomial ring $\mathscr{R}_q = \mathbb{Z}[X]/(q, X^n + 1)$
(k, l)	Dimension of the public matrix $\mathbf{A} \in \mathscr{R}_q^{k \times l}$
τ	Number of ± 1's in challenge c
d	Amount of bits dropped from the public key $\mathbf{t}$
η	Secret key range: $\chi_s = \mathscr{U}([-\eta, \eta]^{n(l+k)})$
$(\gamma_1, \gamma_2, \beta)$	Ranges for signature generation/verification ($\beta = \tau \cdot \eta$)
ω	Number of 1's in the hint $\mathbf{h}$
K	Number of parallel protocol repetitions
ν	Expansion factor for the first l coordinates of the randomness
r'	Randomness ball radius: $\chi_{r'} = \{\lfloor(\nu \mathbf{x}_1, \mathbf{x}_2)\rceil \mid (\mathbf{x}_1, \mathbf{x}_2) \leftarrow \mathscr{B}_{\mathscr{R}, l+k}(r')\}$
r	Target ball radius: $\chi_r = \{\lfloor(\nu \mathbf{x}_1, \mathbf{x}_2)\rceil \mid (\mathbf{x}_1, \mathbf{x}_2) \leftarrow \mathscr{B}_{\mathscr{R}, l+k}(r)\}$
M	Rejection parameter (per party) $M = \left(\frac{r'}{r}\right)^{nl}$

Table 4: Parameters used in ML-DSaaS. These are exactly the same as in [10].

C Message-embedding variant (Remark 5)

We discuss the variant in which the salt is embedded in the message rather than in the FIPS 204 context string, as described in Remark 5.

Impact on syntax and correctness. In this variant, the Client constructs the salted message $\mathsf{msg}' = \mathsf{salt}\|\mathsf{msg}'$ and includes it in the client request. Definition 1 expects $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}, \mathsf{ctx}) = 1$ for the same msg given to Client, whereas correctness now holds with respects to msg'. This is accommodated by redefining Client to output msg' as the signed message, and the correctness and unforgeability games (Figures 1 and 2) track msg' accordingly. In particular, Q_{msg} stores salted messages $\mathsf{msg}' = \mathsf{salt}\|\mathsf{msg}$.

Impact on security. The proof of Theorem 1 requires only that μ contains 256 bits of entropy from the salt. Since H is a random oracle, the position of the salt within its input is immaterial; the bound applies unchanged.

Why we present the context-string variant as primary. The context-string approach satisfies the TSaaS syntax of Definition 1 without modification. It also maintains a natural separation: the salt is a protocol-level artifact, distinct from the application's message. For most deployments, the 32-byte salt fits comfortably within the 255-byte limit alongside up to 223 bytes of application context.

D Correctness

Theorem 3 (Full version of Proposition 1). *In order for our scheme to be correct, we make the following assumptions, exactly as in [10]. We introduce a bound B such that for any m_i returned by* $\mathsf{RSSRecover}$ *and* $(\mathbf{u}_1, \mathbf{u}_2) = \sum_{I \in m_i} \mathbf{s}_I \in \mathscr{R}_q^{l+k}$, $||(1/\nu \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)||_2 \leqslant B$ *with overwhelming probability for a uniformly sampled challenge c and over the randomness of the key generation. We assume that $2\varepsilon = I_{1-1/\phi^2}\left(\frac{n(k+l)+1}{2}, \frac{1}{2}\right)$ for some ϕ and ε, where $I_x(a,b)$ is defined in [10, Definition 2.4]. Finally, we assume that $r'^2 \geqslant r + B^2 + 2rB/\phi$.*

Assume $\Pr[\mathsf{HighBits}(\mathbf{w}, 2\gamma_2) = \mathsf{HighBits}(\mathbf{w}', 2\gamma_2)] = o(1)$ *and* $\epsilon = o(1)$. *The* $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ *scheme p-terminates in the ROM for $p = \mathbb{E}_{k \leftarrow \mathsf{Bin}(K,(1-1/M)^T)}[v_k] + o(1)$ where* Bin *is the binomial distribution, and v_k (for $k \in [0, K]$) is the probability that in at least one of the k combinations we have $||\mathbf{z}^{(1)}||_\infty \leqslant \gamma_1 - \beta$ and $||\delta||_\infty \leqslant \gamma_2$ and the number of 1's in $\mathbf{h}$ is less than ω, when $\mathbf{z}$ is sampled from $\sum_{i \in [T]} \chi_{\mathbf{z}}$ and $\delta = c\mathbf{z}^{(2)} - c\mathbf{t}_\perp$.*

E Unforgeability proof

We prove the unforgeability of the $\mathsf{ML\text{-}DSaaS}^{\mathsf{sh}}$ scheme (Theorem 1).

Proof. As usual, we define a series of hybrids in order to progressively simulate the honest parties without their signing keys. Our proof is very similar to the one from [10]. The major differences are in Game_2 and Game_4 where our TSaaS setting comes into play.

Game_1. This the unforgeability game (Definition 3).

Game_2. This game is a significant change from [10], where our TSaaS setting comes into play. We sample a challenge c and its seed $\tilde{c}$ during the pre-processing round, and program $\mathsf{H}(\mu||\mathbf{w}_\top) := \tilde{c}$ and $\mathsf{SampleInBall}(\tilde{c}) := c$ during the signing round. Recall that $\mathbf{w}_\top$ denotes the high bits of the aggregated nonce $\mathbf{w}$, and μ is obtained by hashing the message with the public key and the salt.

More precisely, during each call to OPP, we sample $\tilde{c} \leftarrow \{0,1\}^{512}$ and $c \leftarrow \mathscr{C}$ (where $\mathscr{C}$ is the challenge space), and add $((\mathsf{pp}_i)_{i\in\mathsf{act}\cap\mathsf{honest}}, c, \tilde{c})$ to $\mathsf{Sessions}$ (instead of just $(\mathsf{pp}_i)_{i\in\mathsf{act}\cap\mathsf{honest}}$).

During each call to $\mathsf{OClient}$, we retrieve and remove $((\mathsf{pp}_i)_{i\in\mathsf{act}\cap\mathsf{honest}}, c, \tilde{c})$ from $\mathsf{Sessions}$, where $(\mathsf{pp}_i)_{i\in\mathsf{act}\cap\mathsf{honest}}$ are the same as those given by the adversary. (If such a tuple does not exist in $\mathsf{Sessions}$, then $\mathsf{OClient}$ aborts, exactly as in the original unforgeability game.) Then, $\mathsf{OClient}$ computes μ using the message and the salt, and $\mathbf{w}_\top$ using $(\mathsf{pp}_i)_{i\in\mathsf{act}}$; and programs $\mathsf{H}(\mu||\mathbf{w}_\top) := \tilde{c}$ and $\mathsf{SampleInBall}(\tilde{c}) := c$. If the random oracle programming fails, the adversary wins. It then calls the PSign algorithms for the honest parties, as in the original unforgeability game. The modified OPP and $\mathsf{OClient}$ algorithms can be found in Figure 6.

Note that each $c, \tilde{c}$ that are sampled during an execution of OPP are used at most once to program random oracles, since they are then removed from $\mathsf{Sessions}$. Thus, the view of the adversary differs only if it queried the random oracle (i) H on $\mu||\mathbf{w}_\top$ before it is programmed, or (ii) $\mathsf{SampleInBall}$ on $\tilde{c}$ before it is programmed. The probability that $\mathsf{SampleInBall}$ is queried on $\tilde{c}$ before it is programmed is bounded by $Q_\mathsf{H} Q_s 2^{-256}$: there are Q_s programmed $\tilde{c}$, the adversary has Q_H chances to guess one of them, and each guess succeeds with probability 2^{-256}. Likewise, the probability that H is queried on $\mu||\mathbf{w}_\top$ before it is programmed is bounded by $Q_\mathsf{H} Q_s 2^{-256}$, since μ has 256 bits of entropy thanks to the salt. Thus,

$$|\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_1} - \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_2}| \leqslant Q_\mathsf{H} Q_s 2^{-255}$$

Game_3. This game is similar to Game_5 from [10]. We ensure that H never returns twice the same $\tilde{c}$ on two different messages (otherwise the adversary wins). This is formalized in Figure 7. By the union bound, we get

$$|\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_2} - \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_3}| \leqslant Q_\mathsf{H}^2 2^{-512}$$

Game_4. In this hybrid, the responses $\mathbf{z}_i$ are sampled in advance during OPP. More precisely, OPP still samples $\tilde{c}$ and c at random, and queries the honest pre-processing algorithms. These sample $\mathbf{z}_i \leftarrow \mathsf{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_\mathbf{z}, \chi_\mathbf{r}, M; \mathbf{r}_i)$ using

OPP(act)

```
assert |act| = T
c̃ ← {0,1}^256
c ← 𝒞
for i ∈ act ∩ honest
    (pp_i, st_i) ← PP(i, st_i)
Sessions := Sessions ∪ {((pp_i)_{i∈act∩honest}, c, c̃)}
return (pp_i)_{i∈act∩honest}
```

OClient(act, msg, $(\mathsf{pp}_i)_{i\in\mathsf{act}}$)

```
Q_msg := Q_msg ∪ {msg}
assert |act| = T
assert ((pp_j)_{j∈act∩honest}, ·, ·) ∈ Sessions
Sessions := Sessions\{((pp_i)_{i∈act∩honest}, c, c̃)}  ▷ retrieve and remove any arbitrary
    tuple of the form ((pp_i)_{i∈act∩honest}, –, –) from Sessions
cr ← Client(msg, act, (pp_i)_{i∈act})
μ := H(tr||0x00||cr.salt||msg)
w := Σ_{i∈act} w_i
w_⊤ := HighBits(w, 2γ_2)
H(μ||w_⊤) := c̃
SampleInBall(c̃) := c
for i ∈ act ∩ honest
    (sig_i, st_i) ← PSign(vk, cr, i, sk_i, st_i)
return (cr, (sig_i)_{i∈act∩honest})
```

Fig. 6: Game_2 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted. If the programming of a random oracle fails, the adversary wins.

H(str)

```
if H(str) ≠ ⊥  return H(str)
c̃ ← {0,1}^256
H(str) := c̃
if ∃str' ≠ str, H(str) = H(str')
    return ⊥  ▷ adversary wins
```

Fig. 7: Game_3 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

the rejection sampling algorithm, as is normally done in the second round. OPP adds these $\mathbf{z}_i$'s in $\mathsf{Sessions}$: sessions are now of the form $((\mathsf{pp}_i)_{i\in\mathsf{act}\cap\mathsf{honest}}, c, \tilde{c}, (\mathbf{z}_i)_{i\in\mathsf{act}\cap\mathsf{honest}})$. In $\mathsf{OClient}$, the oracle uses these responses for the honest parties' signatures, instead of calling the honest parties' partial signature algorithms. The modified PP, OPP and $\mathsf{OClient}$ algorithms can be found in Figure 8.

Note that a set of responses $(\mathbf{z}_i)_{i\in\mathsf{act}\cap\mathsf{honest}}$ programmed during OPP is always used at most once, since it is removed from $\mathsf{Sessions}$ once it is used. Moreover, thanks to our use of the set $\mathsf{Sessions}$ and the programming of the random oracles introduced in Game_3, the c used for computing the $\mathbf{z}_i$'s by OPP in this game is the same as the c used for computing the $\mathbf{z}_i$'s by $\mathsf{OClient}$ in the previous game. Thus, the view of the adversary is unchanged:

$$\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_3} = \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_4}$$

Game_5. This game is similar to Game_7 in [10]. During pre-processing, the honest parties first attempt to compute $\mathbf{z}_i \leftarrow \mathsf{Rej}(c\cdot\mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$, and in case of rejection, sample it from the rejection distribution: $\mathbf{z}_i \leftarrow (\mathbf{z}|\mathsf{Rej})_{v=c\cdot\mathbf{s}_i^{\mathsf{part}}}$. Then, they derive $\mathbf{w}_i = \begin{bmatrix}\mathbf{A}\ \mathbf{I}\end{bmatrix}\cdot\mathbf{z}_i - c\cdot\mathbf{t}_i^{\mathsf{part}}$, where $\mathbf{t}_i^{\mathsf{part}} = \begin{bmatrix}\mathbf{A}\ \mathbf{I}\end{bmatrix}\cdot\mathbf{s}_i^{\mathsf{part}}$. This is depicted in Figure 9. This is perfectly equivalent to the previous game:

$$\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_4} = \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_5}$$

Game_6. This change is identical to Game_8 in [10]. We ensure that the values $c\cdot\mathbf{s}_i^{\mathsf{part}}$ have a twisted norm at most B: whenever this is not the case, the adversary wins. This is depicted in Figure 10. By assumption on B, for any choice of act this is true with overwhelming probability. By the union-bound, we get

$$|\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_5} - \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_6}| \leqslant Q_s \cdot \binom{N}{T} \cdot \mathsf{negl}(\kappa)$$

Game_7. This change is identical to Game_9 in [10]. In case of an abort, we replace $\mathbf{w}_i$ by a uniform value in $\mathscr{R}_q^k$. This is depicted in Figure 11. Using exactly the same analysis as in [10], we obtain that there exists adversaries $\mathscr{B}_1$ and $\mathscr{B}_2$ against the $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{r}}}$ and $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{z}}}$ problems respectively such that

$$\begin{aligned}|\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_6} - \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_7}| \leqslant Q_s \frac{M}{M-1}\Big(\mathsf{Adv}_{\mathscr{B}_1}^{\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{r}}}} + \mathsf{Adv}_{\mathscr{B}_2}^{\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{z}}}}\Big)\\ + Q_s \frac{M}{M-1}\cdot\frac{2\epsilon}{1-\epsilon} + \mathsf{negl}(\kappa)\end{aligned} \quad (2)$$

Game_8. This change is identical to Game_{10} in [10]. We replace the rejection sampling $\mathbf{z}_i \leftarrow \mathsf{Rej}(c\cdot\mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$ by the ideal functionality $\mathbf{z}_i \leftarrow \mathsf{Ideal}(\chi_{\mathbf{z}}, M)$

$\mathsf{PP}(i,\ c\ )$ ▷ for honest parties

$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A}\ \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{z}_i \leftarrow \mathsf{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r}_i)$
if $\mathbf{z}_i = \perp$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \perp)$
$\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
return$(\mathsf{pp}_i := \mathbf{w}_i,\ \mathsf{sig}_i := \mathbf{z}_i^{(1)}\ )$

$\mathsf{OPP}(\mathsf{act})$

assert $|\mathsf{act}| = T$
$\tilde{c} \leftarrow \{0,1\}^{256}$
$c \leftarrow \mathscr{C}$
for $i \in \mathsf{act} \cap \mathsf{honest}$
  $(\mathsf{pp}_i,\ \mathsf{sig}_i\ ) \leftarrow \mathsf{PP}(i,\ c\ )$
$\mathsf{Sessions} := \mathsf{Sessions} \cup \{((\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}, c, \tilde{c},\ (\mathsf{sig}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}\ )\}$
return $(\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}$

$\mathsf{OClient}(\mathsf{act}, \mathsf{msg}, (\mathsf{pp}_i)_{i \in \mathsf{act}})$

$Q_{\mathsf{msg}} := Q_{\mathsf{msg}} \cup \{\mathsf{msg}\}$
assert $|\mathsf{act}| = T$
assert $((\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}, \cdot, \cdot,\ \cdot\ ) \in \mathsf{Sessions}$
$\mathsf{Sessions} := \mathsf{Sessions} \backslash \{((\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}, c, \tilde{c},\ (\mathsf{sig}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}\ )\}$ ▷ retrieve and remove any arbitrary tuple of the form $((\mathsf{pp}_i)_{i \in \mathsf{act} \cap \mathsf{honest}}, -, -)$ from Sessions.
$\mathsf{cr} \leftarrow \mathsf{Client}(\mathsf{msg}, \mathsf{act}, (\mathsf{pp}_i)_{i \in \mathsf{act}})$
$\mu := \mathsf{H}(\mathsf{tr}||0x00||\mathsf{cr.salt}||\mathsf{msg})$
$\mathbf{w} := \sum_{i \in \mathsf{act}} \mathbf{w}_i$
$\mathbf{w}_\top := \mathsf{HighBits}(\mathbf{w}, 2\gamma_2)$
$\mathsf{H}(\mu||\mathbf{w}_\top) := \tilde{c}$
$\mathsf{SampleInBall}(\tilde{c}) := c$
return $(\mathsf{cr}, (\mathsf{sig}_i)_{i \in \mathsf{act} \cap \mathsf{honest}})$

Fig. 8: Game_4 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

$\mathsf{PP}(i, c)$ ▷ **for honest parties**

$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{t}_i^{\mathsf{part}} = [\mathbf{A}\ \mathbf{I}] \cdot \mathbf{s}_i^{\mathsf{part}}$
$\mathbf{z}_i \leftarrow \mathsf{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$
if $\mathbf{z}_i = \perp$
  $\mathbf{z}_i' \leftarrow (\mathbf{z}|\mathsf{Rej})_{v=c\cdot\mathbf{s}_i^{\mathsf{part}}}$
  $\mathbf{w}_i = [\mathbf{A}\ \mathbf{I}] \cdot \mathbf{z}_i' - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \perp)$
else
  $\mathbf{w}_i = [\mathbf{A}\ \mathbf{I}] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \mathsf{sig}_i := \mathbf{z}_i^{(1)})$

Fig. 9: Game_5 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

$\mathsf{PP}(i, c)$ ▷ **for honest parties**

$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{t}_i^{\mathsf{part}} = [\mathbf{A}\ \mathbf{I}] \cdot \mathbf{s}_i^{\mathsf{part}}$
$(\mathbf{u}_1, \mathbf{u}_2) := \mathbf{s}_i^{\mathsf{part}}$
if $||(\nu c\mathbf{u}_1, c\mathbf{u}_2)||_2 > B$
  abort ▷ adversary wins
$\mathbf{z}_i \leftarrow \mathsf{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$
if $\mathbf{z}_i = \perp$
  $\mathbf{z}_i' \leftarrow (\mathbf{z}|\mathsf{Rej})_{v=c\cdot\mathbf{s}_i^{\mathsf{part}}}$
  $\mathbf{w}_i = [\mathbf{A}\ \mathbf{I}] \cdot \mathbf{z}_i' - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \perp)$
else
  $\mathbf{w}_i = [\mathbf{A}\ \mathbf{I}] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \mathsf{sig}_i := \mathbf{z}_i^{(1)})$

Fig. 10: Game_6 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

$\mathsf{PP}(i, c)$ ▷ for honest parties

$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{t}_i^{\mathsf{part}} = \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{s}_i^{\mathsf{part}}$
$(\mathbf{u}_1, \mathbf{u}_2) := \mathbf{s}_i^{\mathsf{part}}$
if $||(\nu c \mathbf{u}_1, c\mathbf{u}_2)||_2 > B$
  abort ▷ adversary wins
$\mathbf{z}_i \leftarrow \mathsf{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$
if $\mathbf{z}_i = \perp$
  $\mathbf{w}_i \leftarrow \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \perp)$
else
  $\mathbf{w}_i = \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \mathsf{sig}_i := \mathbf{z}_i^{(1)})$

Fig. 11: Game_7 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

which doesn't depend on the signing key $\mathbf{s}_i^{\mathsf{part}}$. This is depicted in Figure 12. Using exactly the same Rényi divergence analysis as in [10], we get

$$\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_7} \leqslant \left(1 + \frac{\epsilon}{M-1}\right)^{Q_s} \cdot \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_8}$$

Game_9. This change is identical to Game_{11} in [10]. We remove the abort condition on the norm of $c \cdot \mathbf{s}_i^{\mathsf{part}}$ introduced in Game_6, which can only increase the adversary's advantage:

$$\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_8} \leqslant \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_9}$$

Game_{10}. This change is identical to Game_{12} in [10]. We replace the public key by the rounding of a uniform value, i.e. $\mathsf{Power2Round}(\mathbf{t}, d)$ where $\mathbf{t} \leftarrow \mathscr{R}_q^k$. To preserve consistency, we fix an index I_0 such that $I_0 \subseteq \mathsf{honest}$, and set $\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$. This is depicted in Figure 13. This hybrid is indistinguishable from the previous one by the MLWE assumption, since honest parties no longer use their secret key at this point. Formally, there exists an adversary $\mathscr{B}_3$ against the $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{s}}}$ problem such that:

$$|\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_9} - \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_{10}}| \leqslant \mathsf{Adv}_{\mathscr{B}_3}^{\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{s}}}}$$

Game_{11}. This change is identical to Game_{13} in [10]. We first sample the high bits of the public key from an $\mathsf{ML\text{-}DSA}$ public key, then we sample the full key

$\mathsf{PP}(i, c)$ ▷ for honest parties

$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{t}_i^{\mathsf{part}} = \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{s}_i^{\mathsf{part}}$
$(\mathbf{u}_1, \mathbf{u}_2) := \mathbf{s}_i^{\mathsf{part}}$
if $||(\nu c \mathbf{u}_1, c\mathbf{u}_2)||_2 > B$
  abort ▷ adversary wins
$\mathbf{z}_i \leftarrow \mathsf{Ideal}(\chi_{\mathbf{z}}, M)$
if $\mathbf{z}_i = \perp$
  $\mathbf{w}_i \leftarrow \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \perp)$
else
  $\mathbf{w}_i = \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \mathsf{sig}_i := \mathbf{z}_i^{(1)})$

Fig. 12: Game_8 of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

conditioned on the value of the high bits. This is depicted in Figure 14. There exists an adversary $\mathscr{B}_4$ against the $\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{s}}}$ problem such that:

$$|\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_{10}} - \mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_{11}}| \leqslant \mathsf{Adv}_{\mathscr{B}_6}^{\mathsf{MLWE}_{q,k,l,\chi_{\mathbf{s}}}}$$

Conclusion. Exactly as in [10], we reduce the unforgeability of ML-DSA to Game_{11}. We design an adversary $\mathscr{B}_5$ against the ML-DSA unforgeability game. $\mathscr{B}_5$ takes as input an ML-DSA public key $\mathsf{pk} = (\rho, \mathbf{t}_\top)$. In the threshold Keygen, it uses these values of ρ and $\mathbf{t}_\top$. Queries to the random oracles H and SampleInBall are lazily passed to the random oracles provided by the ML-DSA unforgeability challenger, except when our simulation programs them otherwise. $\mathscr{B}_5$ forwards any forgery it receives to the ML-DSA unforgeability challenger.

The view of the adversary $\mathscr{A}$ is identically distributed since the ML-DSA random oracles sample images from the same distributions as Game_{11}, and the ML-DSA public key is sampled as $(\rho, \mathbf{t}_\top)$ in Game_{11}.

A valid forgery in Game_{11} will also be a valid forgery for ML-DSA. Indeed, the verification procedure only depends on $(\rho, \mathbf{t}_\top)$ which are chosen identically for the ML-DSA signature and the threshold signature. Also, the challenge random oracles coincide on messages for which forgeries are valid (i.e. messages for which the client request oracle was not called) as a $\tilde{c}$ programmed by Game_{11} will never be used by the ML-DSA oracles thanks to the condition added in Game_3. Hence,

$$\mathsf{Adv}_{\mathscr{A}}^{\mathsf{Game}_{11}} = \mathsf{Adv}_{\mathscr{B}_5}^{\mathsf{UF}}$$

Finally, collecting all the bounds gives the theorem statement.

$\mathsf{Keygen}(\kappa, N, T) \to (\mathsf{vk}, \mathsf{sk})$

$\rho \leftarrow \{0,1\}^{256}$
$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$(\mathbf{s}, (\mathsf{sk}_i)_{i \in [N]}) \leftarrow \mathsf{RSS}(N, T, \chi_{\mathbf{s}})$
$\mathbf{t} \leftarrow \mathscr{R}_q^k$
for $I \subseteq [N]$ s.t. $|I| = N - T + 1$ and $I \neq I_0$
  $\mathbf{t}_I := \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{s}_I$
$\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$
$(\mathbf{t}_\top, \mathbf{t}_\bot) := \mathsf{Power2Round}(\mathbf{t}, d)$
$tr \in \{0,1\}^{256} := \mathsf{H}(\rho || \mathbf{t}_\top)$
return $(\mathsf{vk} := (\rho, \mathbf{t}_\top), \mathsf{sk} := (tr, \mathsf{sk}_i)_{i \in [N]})$

$\mathsf{PP}(i, c)$ ▷ **for honest parties**

$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{cr.act})$
$\mathbf{t}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{t}_I$
$\mathbf{z}_i \leftarrow \mathsf{Ideal}(\chi_{\mathbf{z}}, M)$
if $\mathbf{z}_i = \bot$
  $\mathbf{w}_i \leftarrow \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \bot)$
else
  $\mathbf{w}_i = \left[\mathbf{A}\ \mathbf{I}\right] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathscr{R}_q^l \times \mathscr{R}_q^k$
  return$(\mathsf{pp}_i := \mathbf{w}_i, \mathsf{sig}_i := \mathbf{z}_i^{(1)})$

Fig. 13: Game_{10} of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted.

$\mathsf{Keygen}(\kappa, N, T) \to (\mathsf{vk}, \mathsf{sk})$

$\rho \leftarrow \{0,1\}^{256}$
$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$(\mathbf{s}, (\mathsf{sk}_i)_{i \in [N]}) \leftarrow \mathsf{RSS}(N, T, \chi_{\mathbf{s}})$
$(\mathbf{s}, \mathbf{e}) \leftarrow \chi_{\mathbf{s}}$
$\hat{\mathbf{t}} := \mathbf{A}\mathbf{s} + \mathbf{e}$
$(\mathbf{t}_\top, \cdot) := \mathsf{Power2Round}(\hat{\mathbf{t}}, d)$
Sample $\mathbf{t}$ from $\mathscr{U}(\mathscr{R}_q^k)$ conditioned on $(\mathbf{t}_\top, \cdot) = \mathsf{Power2Round}(\mathbf{t}, d)$
for $I \subseteq [N]$ s.t. $|I| = N - T + 1$ and $I \neq I_0$
  $\mathbf{t}_I := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{s}_I$
$\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$
$tr \in \{0,1\}^{256} := \mathsf{H}(\rho || \mathbf{t}_\top)$
return $(\mathsf{vk} := (\rho, \mathbf{t}_\top), \mathsf{sk} := (tr, \mathsf{sk}_i)_{i \in [N]})$

Fig. 14: Game_{11} of our unforgeability proof. The algorithms not depicted in the figure are unchanged. Differences with the previous game are highlighted. It is understood that honest parties now directly use the $\mathbf{t}_I$'s computed during key generation instead of computing them with their secret keys.